Symmetry of performance for a lane merging algorithm for autonomous vehicles

Fabián GIROUD BRISEÑO, Bastien CHOPARD, Pierre LEONE

June 29, 2026

Abstract

We consider the lane merging problem for autonomous cars. We implement a discrete simulation based on the concept where the space is as a collection of cells of arbitrary size and cars are moving from one cell to another. The rules the cars follow, are set by the "speed regulator", which bases the movement on the cars ahead. Once cars reach a point in space called the merging point. The merging algorithm merges cars in an alternating sequence. The goal of this work is to study the consequences of various flows in the bottle neck and try to show that unbalanced flows between lanes have similar performance.

1 Introduction

In this work, we will create a discrete simulation of autonomous cars. We will use the "speed regulator" introduced in the paper *Optimal safe driving dynamics for autonomous interacting vehicles* by Cohen et al. [2]. The goal of this work is to implement the following car system using the speed regulator that enables cars to drive in a safe environment by using a minimum security. Once cars are able to follow each other in their respectively lanes safely, we will make the cars enter an intersection where the merging algorithm will merge both lanes safely. Once the simulations is up and running we will measure the performance of the algorithm for various flow of cars. Initially we measure the performance where the flow of both lanes are equal. Secondly we repeat the simulation but with a flow larger in a lane and then interchange the values of flows to try and determine if unbalance flow rates in lanes have the same performance. The idea of studying unbalanced flows originates from the open questions raised at the end of the paper *Global dynamics of lane merging algorithms for autonomous vehicles* by Bastien Chopard and Pierre Leone [1].

This simulation works on discrete, time, distance and by extension speed, (t$\in \mathbb{N}$, x$\in \mathbb{Z}$, v $\in \mathbb{N}$). Every car is initialized at, $t = 0$, with a position, speed, car size and a lane. For any car at time t, in order to calculate it's state at time $t + 1$ it bases it's calculation on the state of other cars also at time t. Once every car has their state $t + 1$ calculated, the algorithm will update all the cars state from t to $t + 1$. It implies all cars compute their next state at time $t + 1$ using only information from time t, and only after every car has finished computing do they all update together. The system behaves like a parallel update rule also called a synchronous update scheme. It is a discrete-time dynamical system of the form :

$$S_{t+1} = F(S_t) \tag{1}$$

Where S_t is the state of the simulation and F will compute the next state based on that configuration. This method ensures that the same result is obtained across states, regardless of the order in which the cars are processed by the algorithm when computing the next configuration. It also ensures determinism, so that given identical initial conditions, the simulation produces the same result every time, enabling reliable and reproducible analysis.

In this paper, we focus on studying the effects, the lane merging has on the flow of cars at two positions. Before and after the merging point. To ensure reliable measurements we will measure the flow rate of cars in the stationary regimes that appear before and after the merging point. The mathematical definition of a stationary is as a stochastic process whose statistical properties, such as mean and variance, do not change over time. The process is statistically consistent across different time periods. Following the notion of stationary process commonly used in mathematics and statistics, we adapt the definition to our framework as follows.

Definition 1. ***Stationary Regime***
A section of the lane is considered in a stationary regime if all the cars within, follow the following :

1. *All cars are moving at the same maximum speed, (V_{max}), they can not accelerate any more.*
2. *The distance between two cars does not change. Which implies :*
   (a) *The density does not change*
   (b) *The flow does not change*

So when measuring the flow, density, positions and other metrics, the notion of before the merging point refers to the first stationary regime until it's interrupted by the merge algorithm [2.1]. The same logic is applied for the position after the merging point, it means measurements will be done once the merging is stabilized into a stationary regime. The space in between is called the "transient regime". It is characterized by lane-changing activity and fluctuations in flow, speed, densities. In short, every simulation will start in a stationary regime and follow into a non-stationary regime "transient regime" and after sufficient time, the system always converges to a new stationary regime.

In order to be able to replicate and precisely control initial condition. We decided to lay some rules on the characteristics of the initial state to be able to isolate typical patterns.

Proposition 1. *At time, $t = 0$, both lanes start in a stationary regime.*

This enables us to ensure that a stationary regime exists before the merging point.

Proposition 2. *At time, $t = 0$, all cars in either lane will be going at the same V_{max} speed.*

This ensures that once cars are initialized the flow and density will stay the same by preventing cars to accelerate. If the flow were to change, we know it is because of the merging algorithm and cars will be entering the transient regime.

Proposition 3. *At time, $t = 0$, the distance D between any cars must be at least, the security distance D_{secu}.*

$$D \geq D_{secu}$$

$Dsecu$ is given by the speed regulator imported from [2] and detailed in 2.1. This distance is from the nose of the following car to the back of the leading car.

2 Model

In this section, we will explain how the algorithm works, specifically the speed regulator, how it behaves before and after the merging point.
Every car, at time t, is composed of :

$$\text{Car}(t) = \begin{cases} \text{position , } x \\ \text{speed , } v \\ \text{max speed , } V_{max} \\ \text{length , } L \\ \text{lane (0 or 1)} \\ \text{next position , } x_{t+1} \\ \text{next speed , } v_{t+1} \end{cases} \tag{2}$$

The values x_{t+1} and v_{t+1} are computed at time t then stored until every car has been processed to finally be updated when all cars have been processed. It is principle used in (1).

$$\text{t=0 ; } x_{t+1}, v_{t+1}$$
$$\text{t=1 ; } F(x_{t+1}), F(v_{t+1}) = x_t, v_t$$

The model works on discrete time and space. Every time the system is updated a unit of time has passed, also called a "tic". We decided that if one tic is considered equal to 1 second then one unit of distance could be considered 1 meter. Thus, if a car is at position 100, and a tic later is in position 128. The car is traveling with a speed of 28m/s or 100.8km/h, here we will only work with [m/s]. We decided cars to be 5 units long, L = 5m. When a car is said to be at position 100, it means the nose of the car is at 100 and the back of the car goes until 95 if L=5.

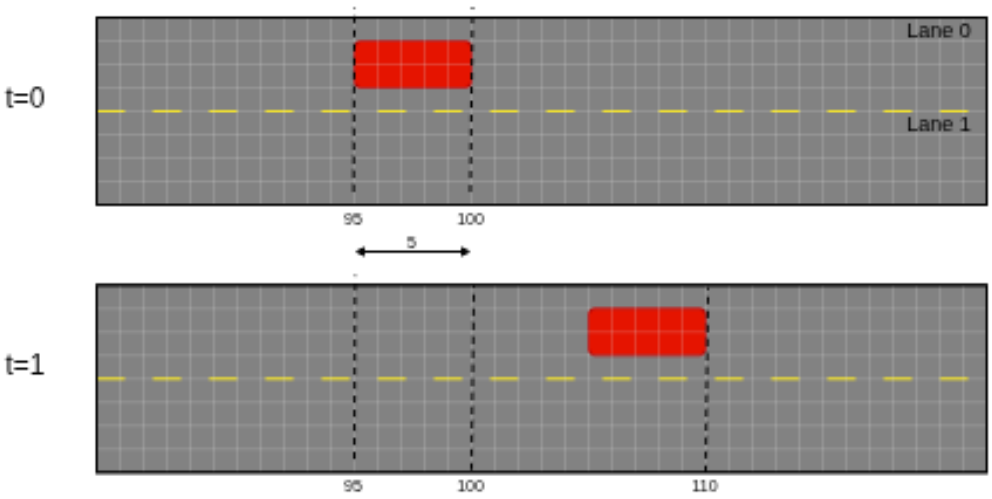


Figure 1: At $t = 0$ the car is at position x=100, with a speed=10m/s, of lenght $L = 5$ on lane 0. The algorithm will compute based on it's position and speed $x_{t+1} = x + v = 100 + 10 = 110$. So one tic later, $t = 1$, x=110, v=10, L=5, $x_{t+1} = 120$

The position and speed of all cars, for the next tic, is given by:

$$x_{t+1} = x_t + v_t \tag{3}$$

$$v_{t+1} = \begin{cases} v_t + 1 \\ v_t \\ v_t - 1 \end{cases} \tag{4}$$

The speed is decided by the speed regulator, which is explained later 2.1. But as shown in (4), this is a simplified version that already tells us that the speed can not change more than one [m/s] per tic. Consequently, the cars can only accelerate at $1m/s^2$ or decelerate at $-1m/s^2$.

We will now explain the logic and structure of the model. Every car has the same structure defined in (2) for every tic of the simulation. All cars are saved in a list and sorted every tic by their position x, in ascending order. When cars get to the merging point the function "Incorporate" looks if cars are allowed to change lanes. Then there is a function for each lane that uses the speed regulator to compute the next state of every car. Once all cars have stored their future state, a function called, "Apply changes", updates (1) the state of every car in the list. This is everything the algorithm does for each tic during the whole simulation. The general structure of the code is given by :

Algorithm 1 General Structure of the model

```
ListCarsLane0 = Initialization Of Cars(gap, NbrCars)
ListCarsLane1 = Initialization Of Cars(gap, NbrCars)
ListCars = ListCarsLane0 + ListCarsLane1
tics = 1000;
for i = 0 to tics do
    Sort(ListCars) //Ascending order
    Incorporation(ListCars, Dmerge)
    ComputeLane0(ListCars)
    ComputeLane1(ListCars)
    ApplyChanges(ListCars)
end for
```

"Initialization of Cars" is the function that initializes all cars, so at all times the simulation has the same amount of cars in the system. The initialization uses a variable "gap" that allows us to add more space between cars, which will later allow us to tune the density and flow of the cars in the lane. "gap" must always be larger or equal to the required security distance, "D_{secu}", that is calculated by the speed regulator 2.1. The function must comply with propositions ; (1) (2) (3). The function initializes the front car, (A), at a position x_A and will initialize the following car, (B), at position :

$$\begin{aligned} x_B &= x_A - (L_A + gap) \\ gap &\geq D_{secu} \end{aligned} \tag{5}$$

With L_A the length or car (A). This will trickle down the road until the function has initialized "NbrCars". To summarize, this function initializes all the cars that will be used for the simulation. All cars start with at their top speed V_{max}, and that

top speed is the same for all cars. It also ensures all cars have at least the minimum security distance between them and allows us to add even more space between cars. As seen in lines 1-3 in (1), "Initialization Of Cars(...)" is used the first time to initialize cars on the lane 0 and then to lane 1 and finally merges the two lists. It allows to create different flow rates and density to study the merging process.

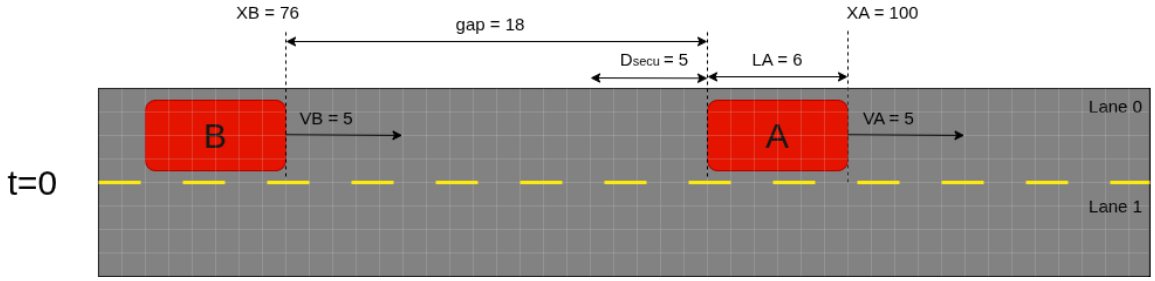


Figure 2: This is an example of how cars are initialized. With initial values for $V_{max} = 5$, $L = 6$, gap = 18 and $x_A = 100$. The speed of V_A must be equal to V_{max} to respect (2). We use (5) to determine the position of car B as, $x_B = 100-(6+18)$ and gap$\geq D_{secu}$, to respect (3). The size of D_{secu} is explained in the following section, (6).

2.1 Speed regulator

The speed regulator is the tools that dictates the change of speed of every car based on the position and speed of the car in front. Since cars can only accelerate and decelerate at $\pm 1m/s^2$ (4), a security distance must be calculated in order to ensure that if the car in front decides to decelerate the car behind has the space and time to slow down as well to avoid a collision. We decided to use the speed regulator introduced in [2]. Here is the algorithm found in the paper :

Algorithm 2 Original Speed Regulator

if $\tilde{d} + df(v_b - 1) \geq df(v_a + 1)$ **then**
    $\tilde{v}_a = v_a + 1$
else if $\tilde{d} + df(v_b - 1) \geq df(v_a)$ **then**
    $\tilde{v}_a = v_a$
else
    $\tilde{v}_a = v_a - 1$
end if

Where "a" is the car in front and "b" the following car, $\tilde{d}$ is the distance from the nose of car "b", to the rear of car "a", and $df(v) = v + (v-1) + (v-2) + ... + 1 = \frac{v(v+1)}{2}$, "The function df gives the braking distance for a car moving at speed v and decelerating constantly." We decided to make some adjustments to the speed regulator to fit our simulation. Because with this version, unobstructed cars will will never stop accelerating. So we decided to add a speed limit V_{max}. We also want to be able to control the slowest speed allowed V_{min}. So theses two modifications gives us the final speed regulator used in the simulation :

Algorithm 3 Speed Regulator + Vmax + Vmin

$\tilde{d} = (x_B - L_B) - x_A$
if $(\tilde{d} + df(v_b - 1) \geq df(v_a + 1))$ **AND** $(v_a + 1 \leq V_{max})$ **then**
    $\tilde{v}_a = v_a + 1$
else if $\tilde{d} + df(v_b - 1) \geq df(v_a)$ **then**
    $\tilde{v}_a = v_a$
else
    if $v_a > V_{min}$ **then**
        $\tilde{v}_a = v_a - 1$
    else
        Car "a" will collide with car "b".
    end if
end if

This algorithm calculates the speed of a car only using values that come from the car ahead and the concerned car. Those five variables are; $(x_A, x_B, V_A, V_B, L_B)$. The algorithm uses them to find the distance between the front of car A to the rear of car B, $\tilde{d} = (x_B - L_B) - x_A$. The speed of the cars, (V_A, V_B), are then used to estimate the braking distance needed for a complete stop, using the function $df(v) = \frac{v(v+1)}{2}$.

We have talked about D_{secu} previously and using the speed regulator we can find what is the minimum safety distance cars should have in a stationary regime (1). Quick reminder, in a stationary regime the speed of all cars are equal. This implies that in the speed regulator $v_a = v_b = v$, and we need to find the minimum value of $\tilde{d}$ to obtain D_{secu}. Since the cars are in a stationary regime, the next speed of car "A" will be the same, $\tilde{v}_a = v_a$. We are in the second condition of the speed regulator, line 4, which is $\tilde{d} + df(v_b - 1) \geq df(v_a)$, and $v_a = v_b = v$:

$$\begin{aligned} \tilde{d} + df(v-1) &\geq df(v) \\ \tilde{d} &\geq df(v) - df(v-1) \\ \tilde{d} &\geq \frac{v(v+1)}{2} - \frac{(v-1)v}{2} \\ \tilde{d} &\geq \frac{v^2+v}{2} - \frac{v^2-v}{2} \\ \tilde{d} &\geq \frac{v^2+v-v^2+v}{2} \\ \tilde{d} &\geq \frac{2v}{2} \\ \tilde{d} &\geq v \end{aligned} \tag{6}$$

So the minimum distance allowed for two cars going same speeds, is v. This result is used in (5) and (3) as D_{secu}, but now we can affirm $D_{secu} = v$. Beware this is only true for cars at constant speed, $v_a = v_b$. This result ensure that the security distance is proportionate to the speed. Here is an illustration of D_{secu} :

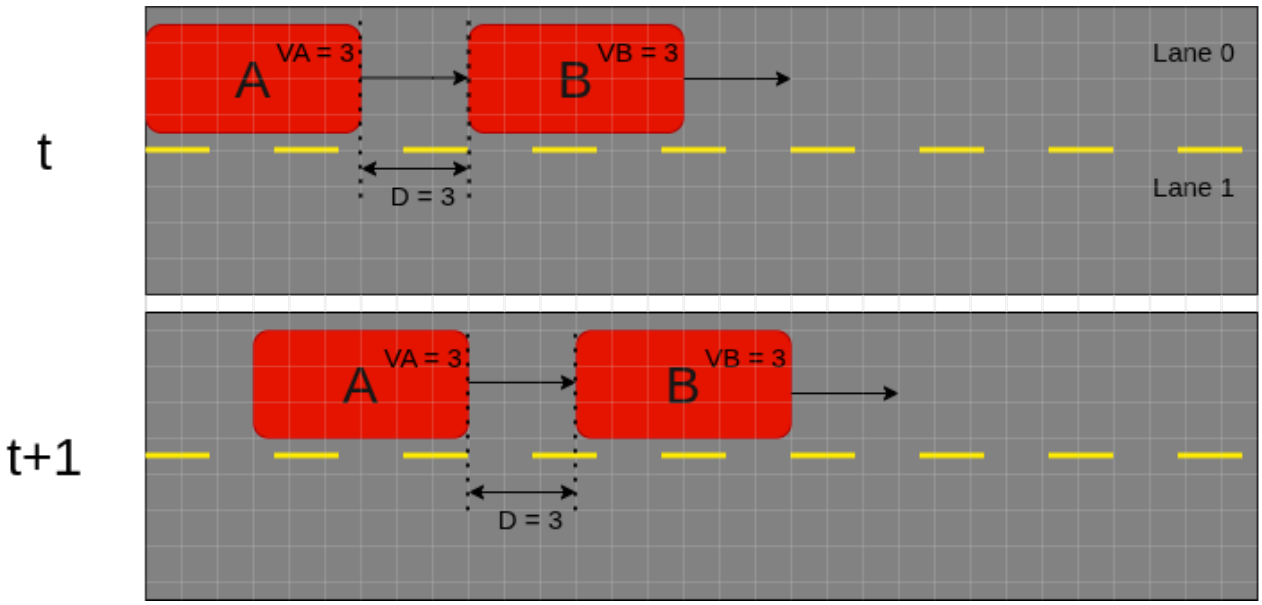


Figure 3: At time t cars are initialized in stationary regime with $V_{max} = v_a = v_b = 3$ and $D = D_{secu} = 3$

Let's analyze the reasoning the speed regulator, (3), makes at time t to compute the next speed of cars A and B for $t+1$. For car B since it has no cars in front, $\tilde{d} = \infty$, the first condition in line 2 will be :

$$\begin{aligned} \tilde{d} + df(v_b - 1) \geq df(v_a + 1)) \textbf{ AND } v_a + 1 \leq V_{max} \\ \infty \geq df(3+1)) \textbf{ AND } 3 + 1 \leq 3 \\ \infty \geq 10 \textbf{ AND } 3 + 1 \leq 3 \end{aligned} \tag{7}$$

We see this condition is false so the speed regulator will evaluate the second one in line 4 :

$$\begin{aligned} \tilde{d} + df(v_b - 1) &\geq df(v_a) \\ \infty + df(3-1) &\geq df(3) \\ \infty &\geq 6 \end{aligned} \tag{8}$$

This time the condition is TRUE, so it will execute line 5, which keeps car "B" with the same speed of $v_b = 3$. Now for car "A", this condition on line 2 will also be false :

$$\begin{aligned} \tilde{d} + df(v_b - 1) \geq df(v_a + 1)) \textbf{ AND } v_a + 1 \leq V_{max} \\ 3 + df(3 - 1) \geq df(3 + 1)) \textbf{ AND } 3 + 1 \leq 3 \\ 3 + 3 \geq 10 \textbf{ AND } 3 + 1 \leq 3 \end{aligned} \tag{9}$$

And the second condition on line 4 will be true, also setting the speed of car A to the same speed, $v_a = 3$:

$$\begin{aligned} \tilde{d} + df(v_b - 1) \geq df(v_a) \\ 3 + df(3 - 1) \geq df(3) \\ 3 + 3 \geq 6 \end{aligned} \tag{10}$$

To summarize, in figure 3 at time t, $v_a = v_b = V_{max} = D_{secu} = 3$ and the distance between them is $\tilde{d} = 3$. When the speed regulator (3), is applied to car "B" the first condition will be false (7), the second condition will be true (8), and keep the same speed. For car "A", the first condition is false (9) and the second condition is true (10), keeping the same speed. So at $t + 1$ the variables, $(v_a, v_b, \tilde{d})$, used by the speed regulator are the same that in time t, keeping the cars in a stationary regime and in the same configuration through space. Cars initialized with the minimum security distance and at V_{max} will keep this distance if there is no interruption.

The following figure shows how cars initialized in a stationary regime will behave when they arrive at the tail of a traffic jam.

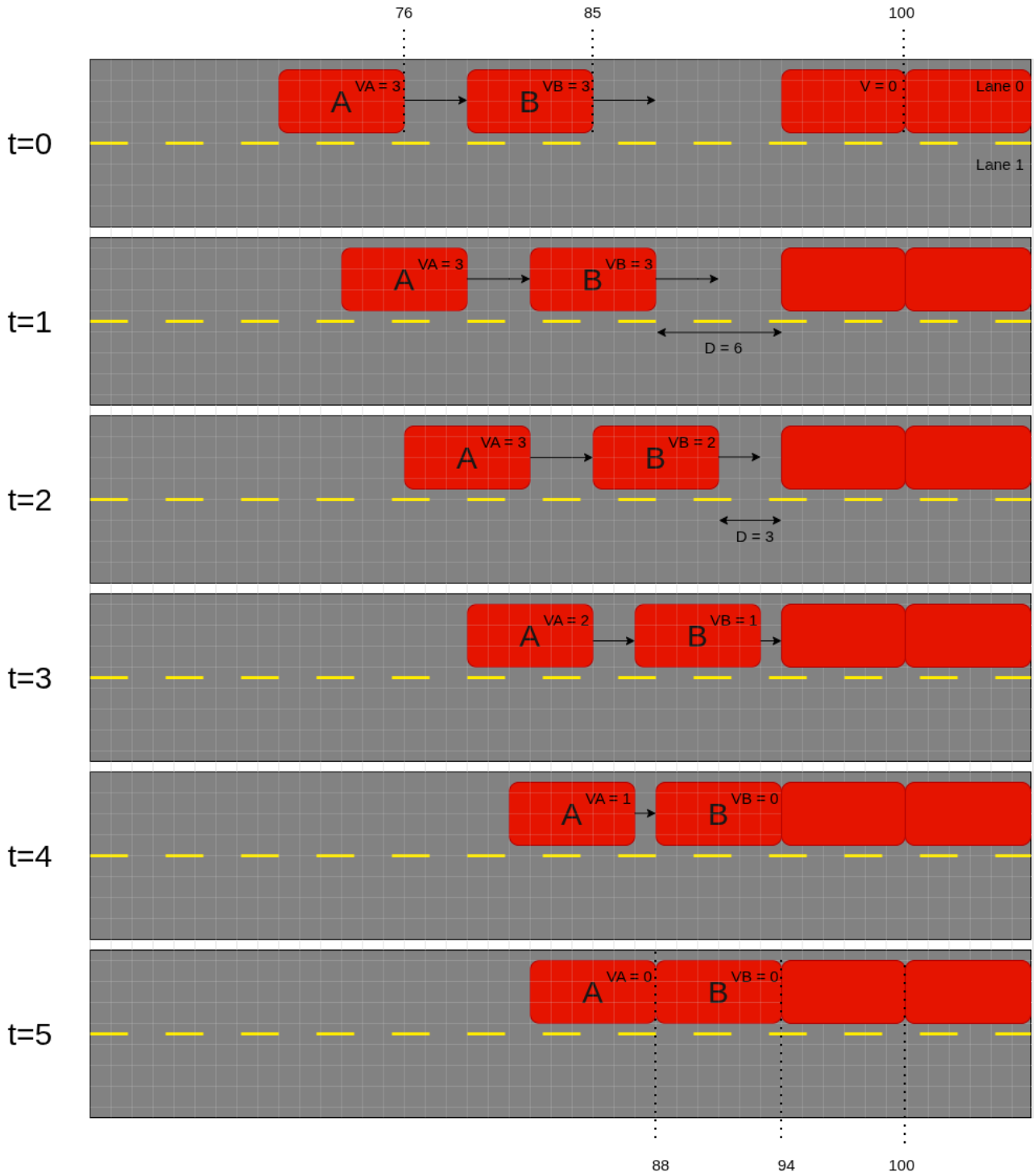


Figure 4: Cars "A" and "B" are initialized in a stationary regime with $V_{max} = 3$ and $D = D_{secu} = 3$. They are arriving at the tail of a traffic jam with immobile cars $V = 0$.

At all times the distance D between cars is equal or higher to D_{secu}. We can observe at $t = 5$ that $D = 0$ because $v_a = v_b = 0$:

$$\begin{aligned} \tilde{d} + df(0-1) &= df(0) \\ \tilde{d} &= df(0) - df(0-1) \\ \tilde{d} &= 0 - 0 \\ \tilde{d} &= 0 \end{aligned} \tag{11}$$

The following figure shows how two cars accelerate from a standing position to $V_{max} = 3$:

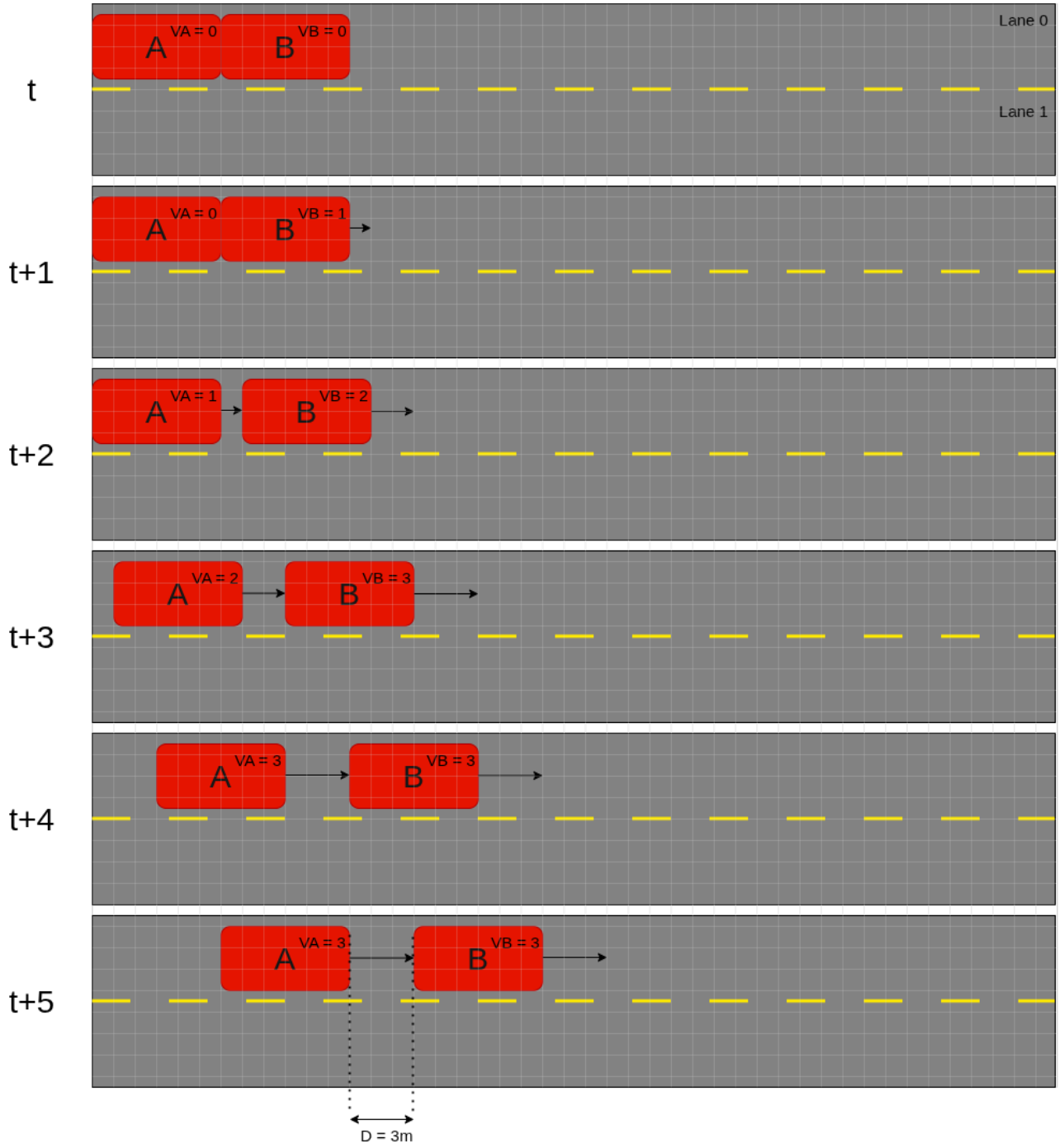


Figure 5: Cars "A" and "B" are initialized with no speed $v_a = v_b = 0$, and will accelerate to $V_{max} = 3$.

At all times the distance D between cars is equal to their speed, $D = V = D_{secu}$ and at $t + 4$ the cars have entered a stationary regime with a distance between them of D_{secu}. In figures 4 and 5 we see cars decelerate and accelerate at a rate of $1m/s^2$, because the speed regulator is applied exactly once per car per second and the speed regulator (3) modifies the speed in lines 3 and 8 by $\pm 1m/s$. This observations follows the definition made in (4).

In figures 3, 4, and 5 cars have either been initialized in a stationary regime or reached it. With each time the distance between cars stabilizing to D_{ecu}. But there are other configuration where the characteristics of the speed regulator show up. The first one is intuitive, if cars are initialized with a distance larger than D_{secu}, the distance will stay the same. As shown in this illustration :

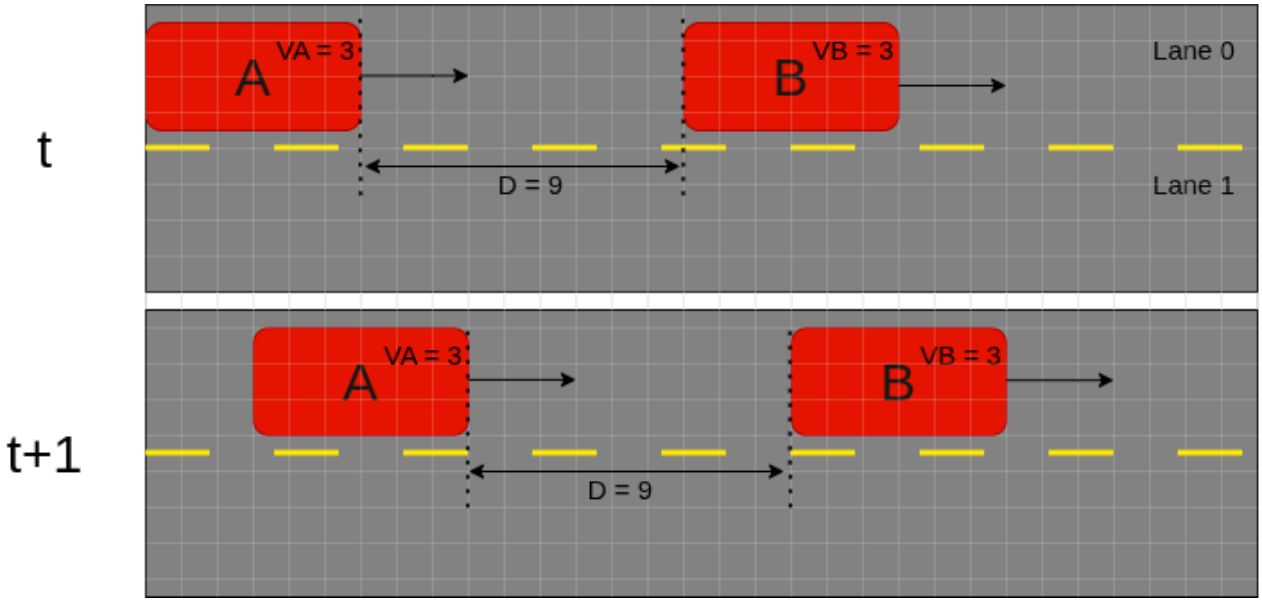


Figure 6: Cars "A" and "B" are initialized in a stationary regime with $V_{max} = 3$ and $D = 9 \geq D_{secu} = 3$.

This scenario is similar to the one in figure 3 since they are also in a stationary regime, but in figure 3 the reason that car "A" could not get closer to "B", was due to the condition in line 2 being false (9). In this scenario is because "A" has reached V_{max} and the first condition does not allow it to have a speed higher than V_{max}. So any cars in a stationary regime will keep their distance D through space, even if it is larger than D_{secu}.
But if car "A" has a V_{max} higher than car "B" it would reduce the distance between them to D_{secu}.

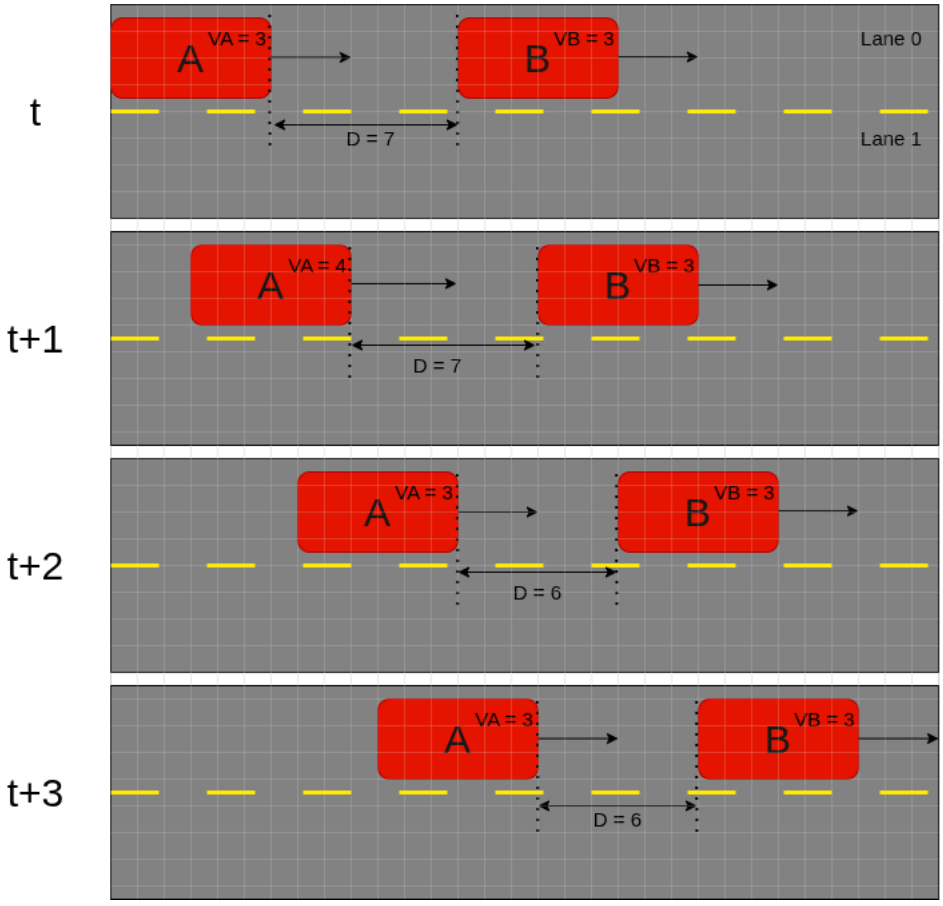


Figure 7: Cars "A" has a max speed of 10, $V_{maxA} = 10$, and at time t, it's speed is 3, $v_a = 3$. Car "B" has a max speed of 3, $V_{maxB} = 3$, and at time t it's speed is 3, $v_b = 3$. The distance between them is 7, $D = 7$. At $t + 3$ the cars are in a stationary regime.

Here car "B" behaves like in figure 3 and 6, staying with a speed of 3 since $v_b = V_{maxB} = 3$. But car "A" can accelerate at time t, since it's current speed is lower than it's maximum speed, ($v_a = 3 \leq V_{maxA} = 10$), and the distance D to the car "B" is smaller than the minimum security distance, $D = 7 \geq 3 = D_{secu}$. This is exactly what happens at $t + 1$, the first condition of the speed regulator (3) is true, increasing the speed of car "A" by one.

$$\begin{aligned} \tilde{d} + df(v_b - 1) \geq df(v_a + 1)) \textbf{ AND } v_a + 1 \leq V_{maxA} \\ 7 + df(3 - 1) \geq df(3 + 1)) \textbf{ AND } 3 + 1 \leq 10 \\ 7 + 3 \geq 3 \textbf{ AND } 3 + 1 \leq 10 \\ v_a = v_a + 1 \end{aligned} \tag{12}$$

But at $t+2$ car "A" does not keep accelerating to reduce the D to D_{secu}, not even keeping the same speed, it decelerates. This means that the conditions of the speed regulator (3), in lines 2 and 4 are both false in order to execute line 8 ($v_a = v_a - 1$). Let's look at the speed regulator for car "A" at $t + 2$, $v_a = 4$, $v_b = 3$ and $D = 6$:

$$\begin{aligned} \tilde{d} + df(v_b - 1) \geq df(v_a + 1)) \textbf{ AND } v_a + 1 \leq V_{maxA} \\ 6 + df(3 - 1) \geq df(4 + 1)) \textbf{ AND } 4 + 1 \leq 10 \\ 6 + 3 \geq 15 \textbf{ AND } 4 + 1 \leq 10 \end{aligned} \tag{13}$$

We see indeed the first condition in line 2 is false, the next step the speed regulator does is the condition on line 4 :

$$\begin{aligned} \tilde{d} + df(v_b - 1) \geq df(v_a) \\ 6 + df(3 - 1) \geq df(4) \\ 6 + 3 \geq 10 \\ 6 \geq 7 \end{aligned} \tag{14}$$

Which is also false, thus the reason it executes line 8 ($v_a = v_a - 1$). This characteristic of the speed regulator means that cars catching up to slower cars will keep a greater security distance than cars that have been initialized with the minimum security distance D_{secu} or cars that accelerated together like in figure 5. We can find at what distance , D, cars in this scenario will stabilize.

We want to to find the distance at which a faster car will stabilize when it catches a slower car. We know that it is larger than D_{secu}. In figure 7 at $t + 1$ car "A" accelerated. We can find what is smallest value of D, before it will not accelerate anymore. Car "A" will not accelerate when ($\tilde{d} < df(v_a + 1) - df(v_b - 1)$) is true.

$$\begin{aligned} \tilde{d} &< df(v_a + 1) - df(v_b - 1) \\ \tilde{d} &< \frac{(v_a + 1)(v_a + 2)}{2} - \frac{(v_b - 1)v_b}{2} \\ \tilde{d} &< \frac{v_a^2 + 3v_a + 2 - v_b^2 + v_b}{2} \end{aligned} \tag{15}$$

And we know that at $t + 1$ the speeds of cars "A" and "B" were the same $v_a = v_b = 3$.

$$\begin{aligned} \tilde{d} &< \frac{v_a^2 + 3v_a + 2 - v_b^2 + v_b}{2} \\ &\textbf{With : } v_a = v_b = v \\ \tilde{d} &< \frac{v^2 + 3v + 2 - v^2 + v}{2} \\ \tilde{d} &< \frac{4v + 2}{2} \\ \tilde{d} &< 2v + 1 \end{aligned} \tag{16}$$

And at $t+1$ the distance between "A" and "B" is $D = 7$, which is bigger than the distance before "A" stops accelerating, $\tilde{d} < 2v + 1 = 2 * 3 + 1 = 7 = D$. This means car "A" was as close as possible to "B" that allowed "A" to accelerate.

We want to find what is the minimum distance "A" can sustain it's speed. Car "A" will decrease it's speed when ($\tilde{d} + df(v_b - 1) < df(v_a)$) is true.

$$
\begin{aligned}
\tilde{d} + df(v_b - 1) &< df(v_a) \\
\tilde{d} &< df(v_a) - df(v_b - 1) \\
\tilde{d} &< \frac{v_a(v_a + 1)}{2} - \frac{(v_b - 1)v_b}{2} \\
\tilde{d} &< \frac{v_a^2 + v_a - v_b^2 + v_b}{2}
\end{aligned} \tag{17}
$$

At $t + 2$ car the conditions are, $(v_a = 4, v_b = 3)$, so in (17) we can find what is the smallest value $\tilde{d}$ must be to sustain it's speed.

$$
\begin{aligned}
\tilde{d} &< \frac{v_a^2 + v_a - v_b^2 + v_b}{2} \\
&\text{We substitute : } v_a = v_b + 1 \\
\tilde{d} &< \frac{(v_b + 1)^2 + v_b + 1 - v_b^2 + v_b}{2} \\
\tilde{d} &< \frac{v_b^2 + 2v_b + 1 + v_b + 1 - v_b^2 + v_b}{2} \\
\tilde{d} &< \frac{4v_b + 2}{2} \\
\tilde{d} &< 2v_b + 1
\end{aligned} \tag{18}
$$

At $t + 2$ the condition to sustain the speed is that $\tilde{d} < 2v_b + 1 = 2 * 3 + 1 = 7$, but $D = 6 < \tilde{d}$. It's the reason car "A" decreased the speed. This implies that if car "A" was at any other position further behind it would have sustained it's speed. If we look at $t + 3$, the values are, $(v_a = v_b = 3, D = 6)$, but the minimum distance allowed to accelerate is $D = 2v + 1$, as shown in (16), "A" is too close to "B" it should slow down to fall to have $D = 2v + 1 = 7$. To find what value should D be decrease it's speed. We take (17) and use the state values at $t = 3$ which are, $(v_a = v_b = 3, D = 6)$.

$$
\begin{aligned}
\tilde{d} &< \frac{v_a^2 + v_a - v_b^2 + v_b}{2} \\
&\text{We use : } v_a = v_b = v \\
\tilde{d} &< \frac{v^2 + v - v^2 + v}{2} \\
\tilde{d} &< \frac{2v}{2} \\
\tilde{d} &< v
\end{aligned} \tag{19}
$$

This result is exactly the same found in (6). This means if "A" wants to decelerate it should have a $D = v = 3$, but at $t = 3$ the value is $D = 6$. Car "A" will not decelerate because it's is not close enough to "B" and will not accelerate because it is too close to "B", it will simply sustain it's speed.

This is the reason why cars catching up on slower cars stabilize at a distance $D = 2v_b$. The car behind, can not accelerate as shows (16), because it needs a distance $D = 2v + 1$. In order to decelerate to be able to reach $D = 2v + 1$ it must have $D = v$, as shown by (19). Which will never happen because $v_a = v_b$. Car "A" will stay at distance $D = 2v$ because it is too close to "B" to acelerate and too far from "B" to slow down.

There is a last configuration that is interesting to analyze, in case two cars are initialized with a distance shorter than the minimum distance required, $D \leq D_{secu}$, the following car will slow down and stabilize in stationary regime with, $D = D_{secu}$.

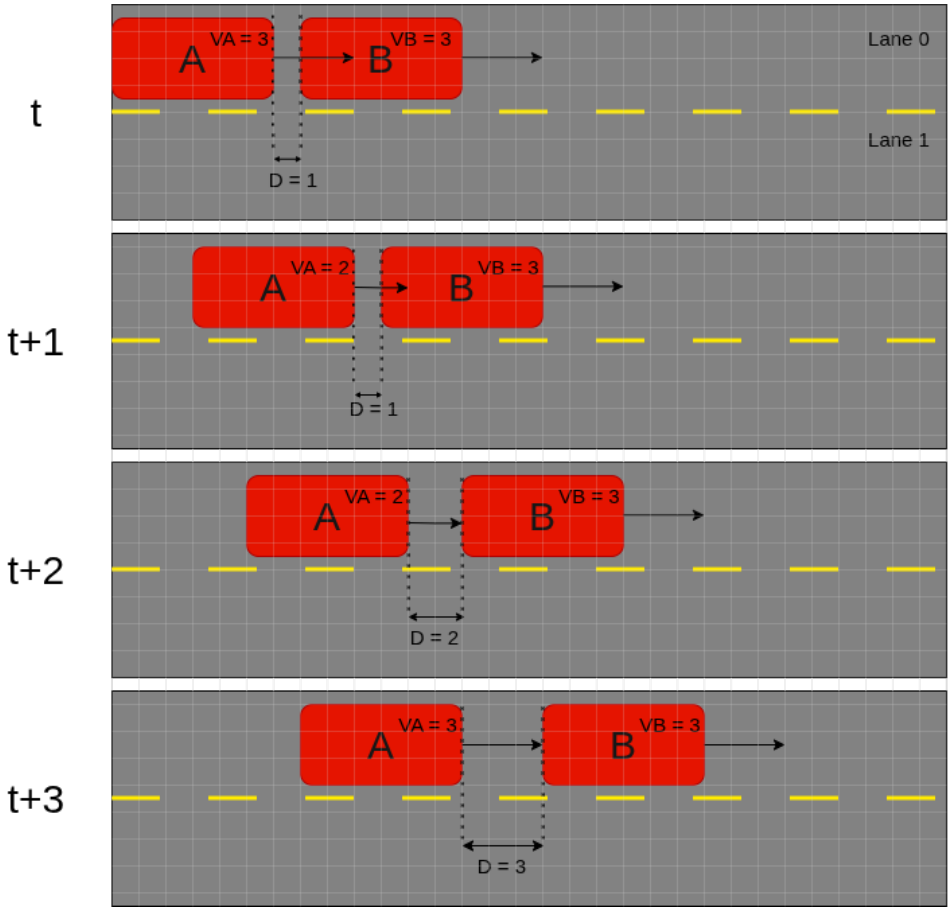


Figure 8: Here cars are initialized with $v_a = v_a = 3$, with the maximum speed of 3, $V_{max} = 3$, and a distance between them of 1, $D = 1$, but the minimum security distance should be 3, $D_{secu} = v_a = 3$.

We see that car "A" will reduce it's speed to enable the gap between them to grow and will speed up again to $V_{max} = 3$ and enter a stationary regime with $D = D_{secu} = 3$. This scenario should never happen because cars are initialized by the function "Initialization Of Cars(..)" in (1), that purposely ensure $D \geq D_{secu}$, see (5). But we will see a very scenario happen in the merging algorithm section 2.2. The cars will not be in the same lane, but all the other conditions will be met. This happens when cars of two different lanes behave as if they were in the same lane. They are virtually in the same lane.

We will summarize and enumerate the characteristics about the speed regulator :

1. Cars compute their speed only based on the car in front and it's self. $(v_a, x_a, v_b, x_b, L_b)$
2. Cars can only accelerate at $\pm 1 m/s^2$, (4).
3. The minimum security distance D_{secu}, is proportional to the speed, $D_{secu} = V$, (6).
4. At speed $v = 0$, $D_{secu} = 0$, meaning the nose of car "A" is touching the rear end of car "B", figure 4 and (11).
5. Cars initialized at a halt, if unobstructed will get to V_{max} and the distance between them will be D_{secu}, figure 5.
6. Cars will only slow down if they catch up to a slower car in order to respect D_{secu}, figure 4.
7. If two cars are initialized at the same V_{max}, the distance between them will stay the same if it is $\geq D_{secu}$, figure 3 and 6.
8. If car "A" catches up car "B", the distance between them will stabilize at $2v_b$ and the speed will be $v_a = v_b$, figure 7 and (19), (16).
9. If cars "A" and "B" are initialized with a distance between them smaller than D_{secu}, they following car will slow down and enter a stationary regime with the distance between "A" and "B" equal to d_{secu}. See figure 8.

2.2 Merging Algorithm

In this section we will explain how the merging is done by giving simple examples and slowly increasing the complexity until the last example where we merge two full lanes.
If we take the general structure of the code showed in (1), there is no specific function in charge of the merging. It is handled by three functions; Incorporation(ListCars, D_{merge}), ComputeLane0(ListCars), ComputeLane1(ListCars), in lines 7,8,9. The functions "ComputeLane0-1" are responsible of applying the speed regulator on the cars in their respective lanes. First a quick reminder, when the speed regulator is applied to a car it only uses the variables of the car concerned and the the one directly in front, (1). This means cars behind the merging point, will apply the speed regulator on them and the car in front of them, both cars must be in **the same lane**. "ComputeLane0(..)" will only modify the speed and positions of cars in lane 0 and "ComputeLane1(..)" will only modify the speed and positions of cars in lane 1, as long cars have not reached the merging point. What happens on a lane will have no impact on the other one.
Once a car reaches the merging point, it will look in the sorted list of cars, "ListCars", which car is directly in front, but this time it can be any car, not necessarily a car on the same lane. Since the list of cars, "ListCars", is sorted independently of the lane, the car directly in front is at index "i+1". This algorithm virtually puts all cars in the the same lane, and thanks to the characteristics of the speed regulator seen at, (2.1), the gap between the two cars will be guaranteed to either D_{secu} or $2D_{secu}$, even if they are not in the same lane. For cars in lane 1, when they have passed the merging point, the function "Incorporation(ListCars, D_{merge})" will monitor the distance between the car directly in front, D_{front}, and the distance with the car directly behind, D_{behind}. When these two gaps, (D_{front}, D_{behind}), have reached the size of, D_{merge}, the function "Incorporation(ListCars, D_{merge})" will switch the car from lane 1 to lane 0.
This may not seem clear explained with only words so here are various scenarios implementing the merging algorithm.

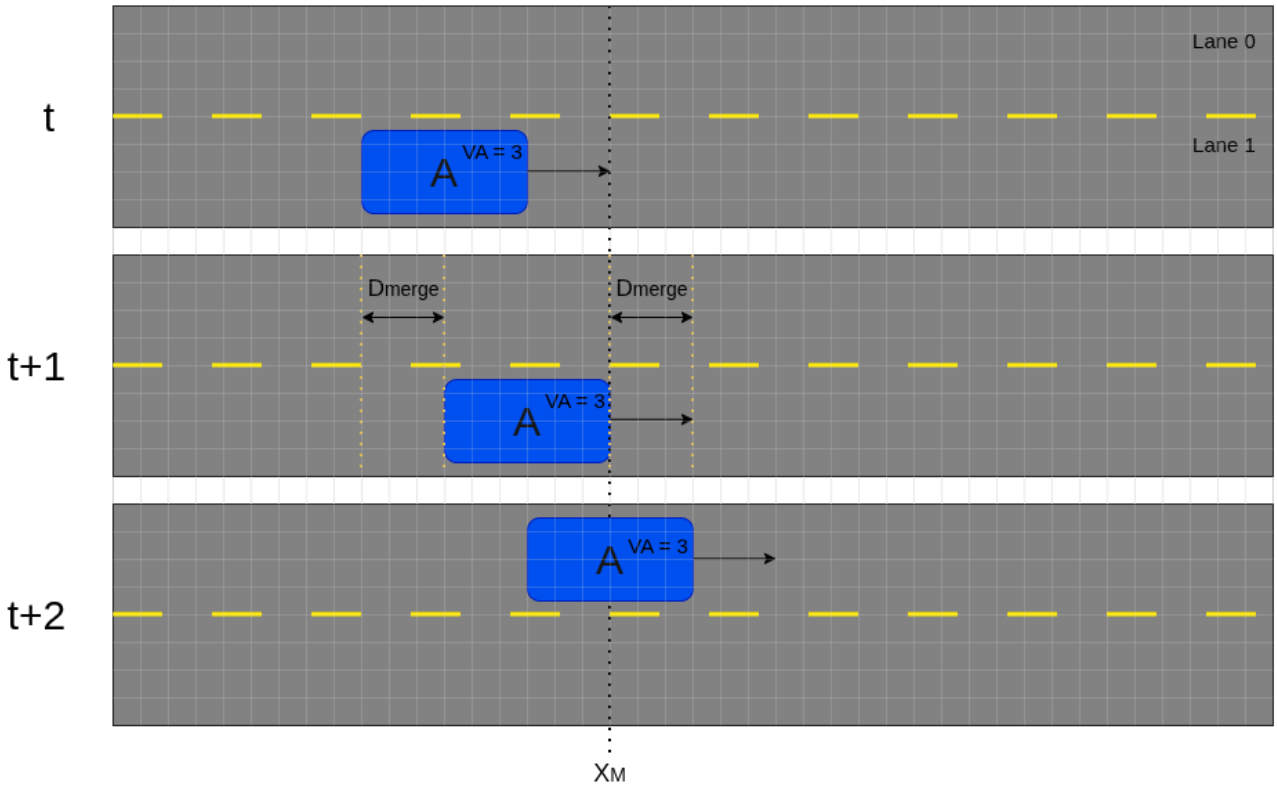


Figure 9: Car "A" is initialized at $v_a = V_{max} = 3$ in lane 1 behind the merging point and the variable D_{merge} used in the function "Incorporation(ListCars, D_{merge})", is equal to 3. $D_{merge} = V_{max} = 3$

This scenario is the simplest possible with a single car initialized in lane 1, to it's max speed. At time $t + 1$ the function "Incorporation(ListCars, D_{merge})" evaluates if the distance, D_{merge}, in front and behind is clear of any cars. It is indeed clear of any obstacles so at $t+2$ the car merges into lane 0. If car "A" where to be initialized in lane 0, arriving at the merging point, x_M, the function "Incorporation(..)" would not be triggered since it is only used to merge cars from lane 1 to lane 0. And car "A" would continue at it's max speed in lane 0 indefinitely.

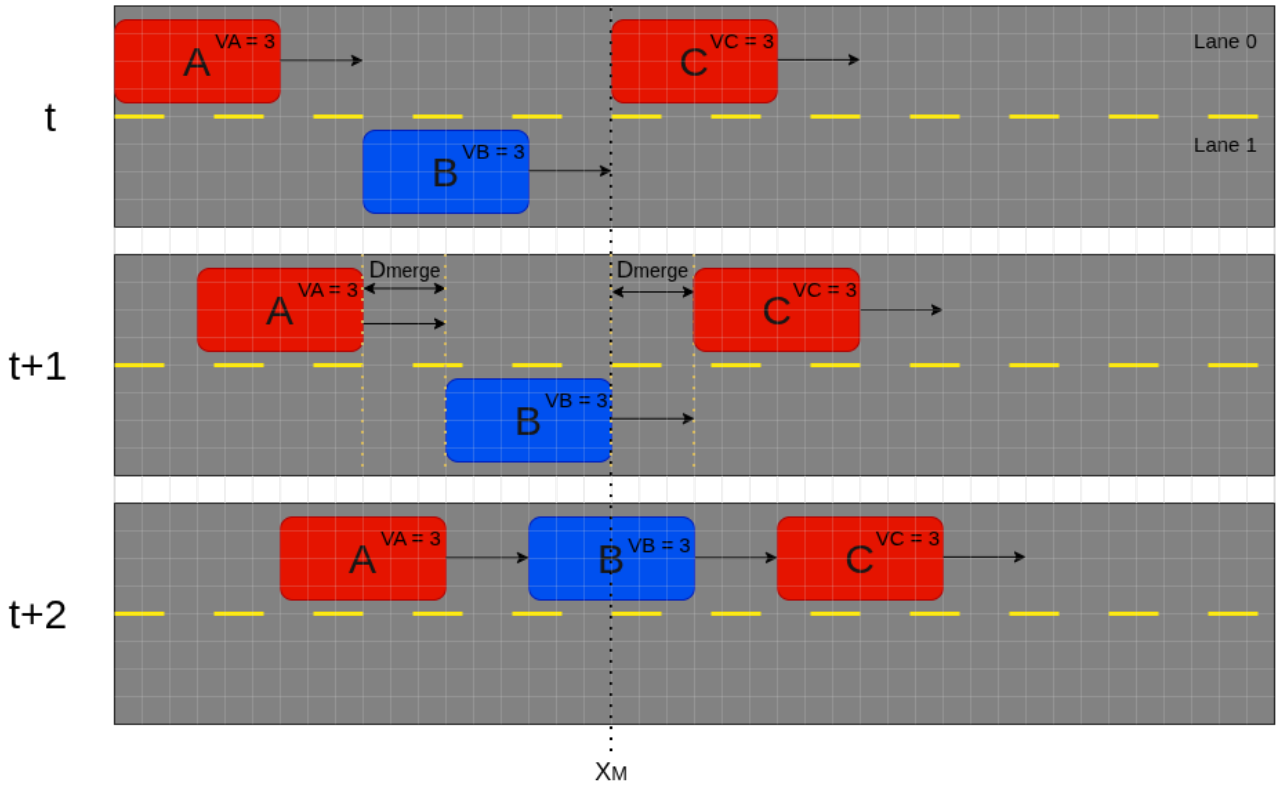


Figure 10: All the cars are initialized at the max speed, $V_{max} = 3$ and $D_{merge} = 3$.

Here the merging of car "B" does not impact the speed of the two other cars. At $t + 1$ "A" is sufficiently far behind "B" to be outside the distance D_{merge}. Car "C" is s sufficiently far in front of "B" to be outside the distance D_{merge}. At $t + 3$ the three cars enter a stationary regime withe the distance between them equal to D_{secu}. We notice that in this scenario car "B" behaves exactly like car "A" in figure 9, where the car was alone.

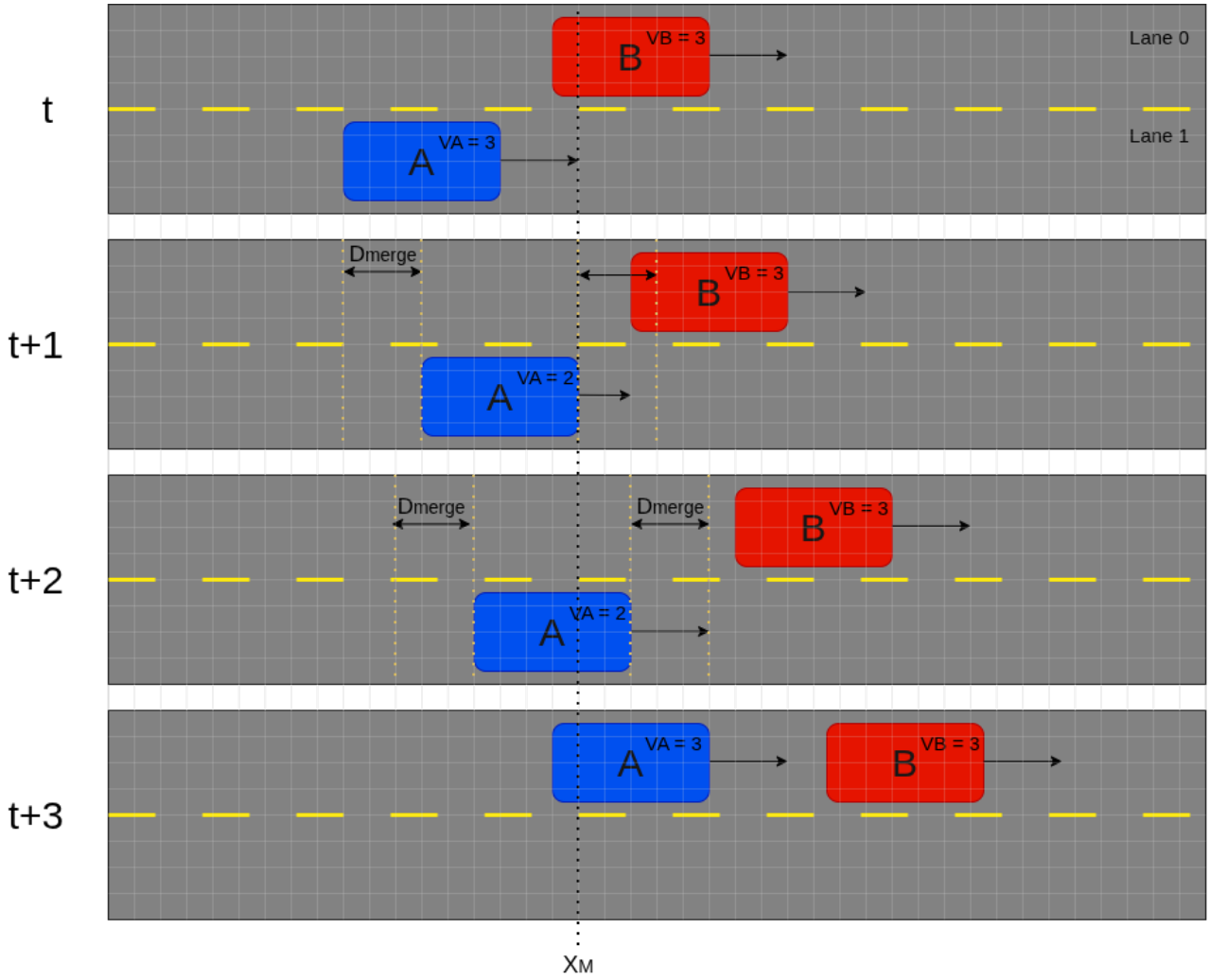


Figure 11: Both cars are initialized at their maximum speed of $V_{max} = 3$ and $D_{merge} = 3$.

In this configuration, car "B" is not far enough in front, so at $t + 1$ car "A" must slow down to increase the distance between them. At $t + 2$ the gap is large enough and "A" can merge onto lane 0. At $t + 3$ they enter a stationary regime and the distance between them is larger than D_{secu}.

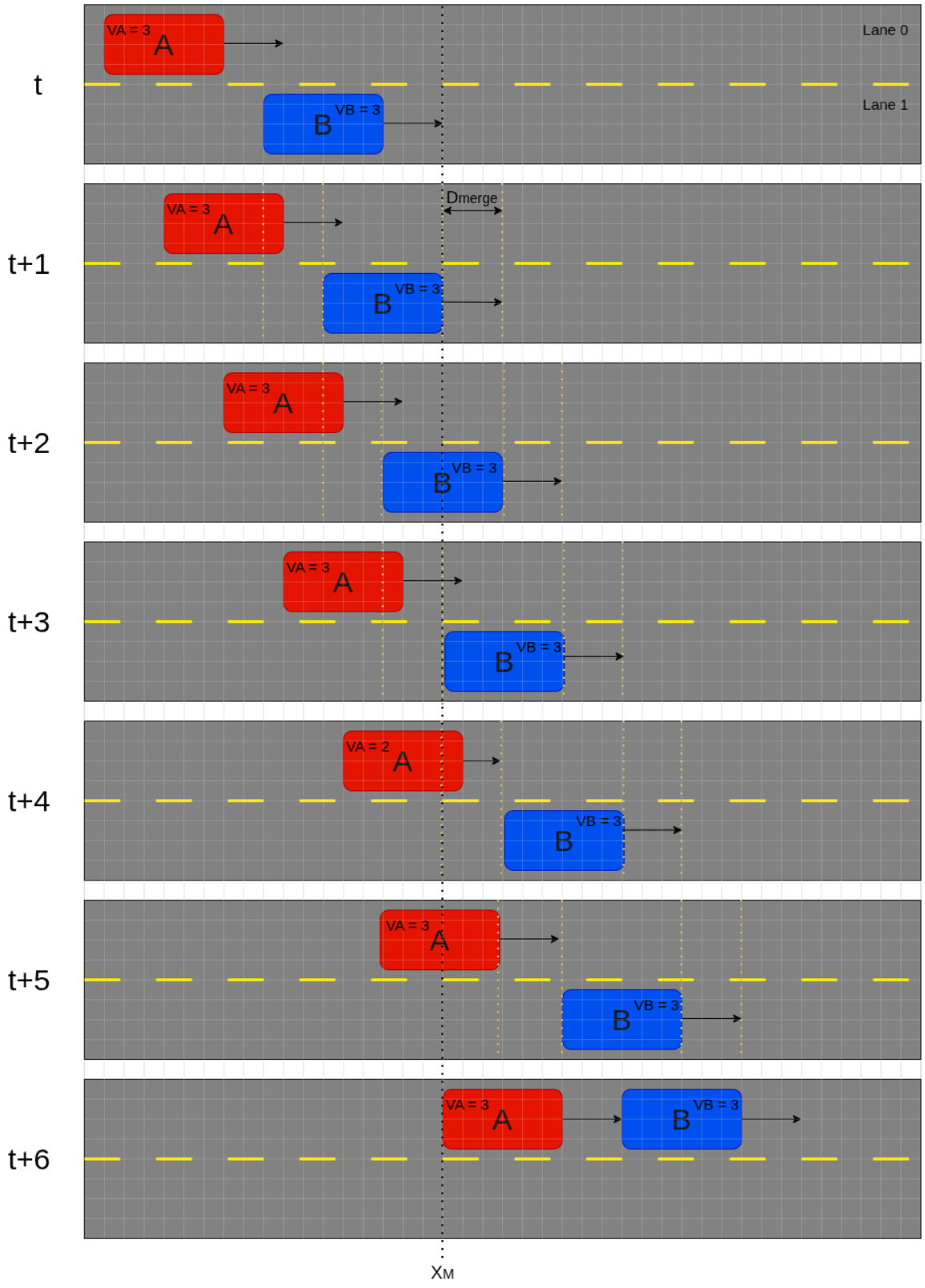


Figure 12: Both cars are initialized at their maximum speed of $V_{max} = 3$ and $D_{merge} = 3$.

This is a similar scenario as the previous one but instead of the car in front being too close, it's the car behind that is too close. The major difference is that it takes twice the time to merge. Because when car "B" gets to the merging point it is not able to increase the gap between them for two reasons. The first being that "B" is already at V_{max}. But even if that was not the case, a car bases it's speed on the car in front, (1). So what happens behind car "B" has no impact on it's speed. It must wait for car "A" to reach x_M, so "A" can slow down and increase the distance between them to reach D_{merge}. At $t + 6$ they enter a stationary regime with the distance between them equal to D_{secu}.

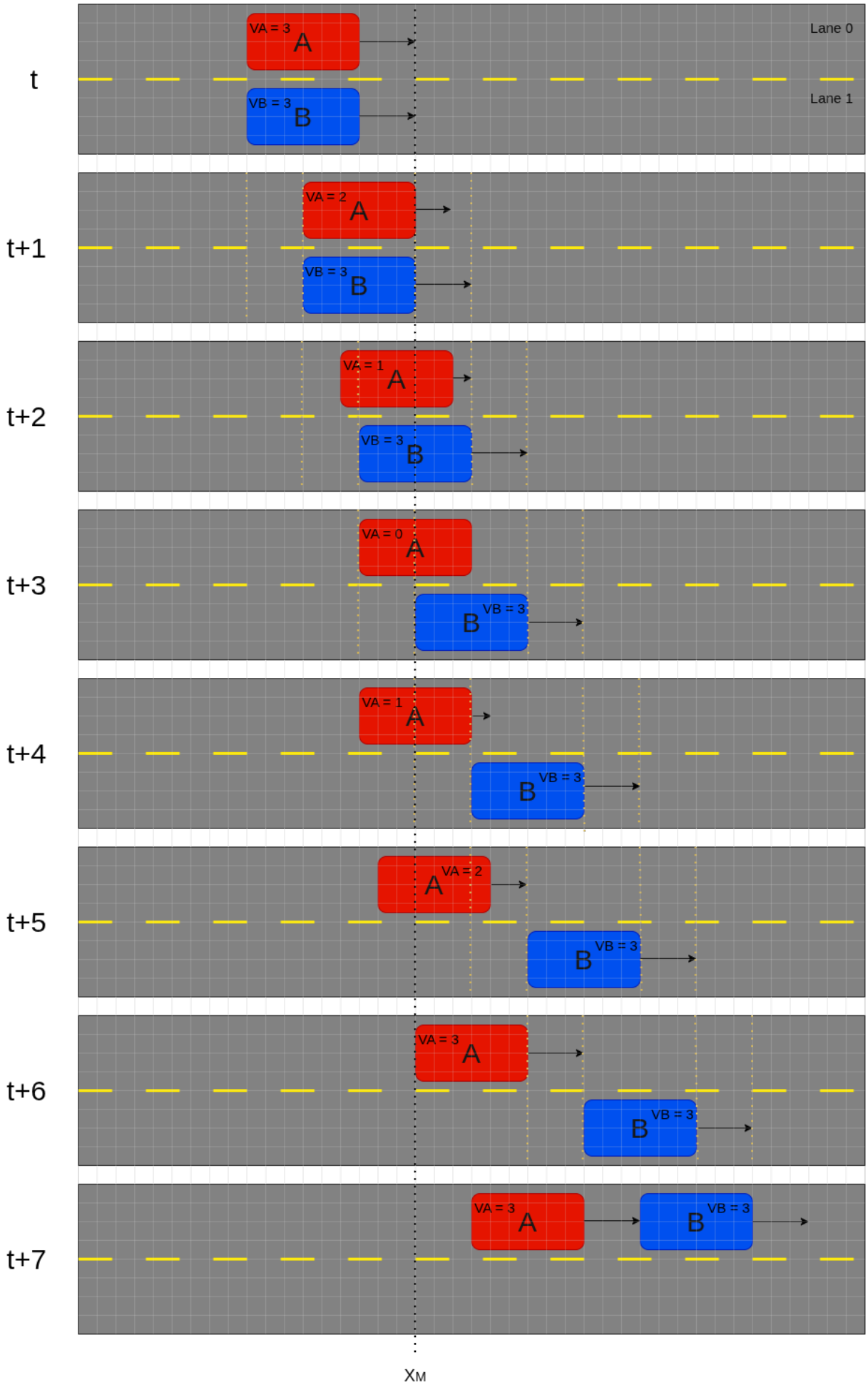

t
t+1
t+2
t+3
t+4
t+5
t+6
t+7
Lane 0
Lane 1
VA = 3
A
VB = 3
B
VA = 2
A
VB = 3
B
VA = 1
A
VB = 3
B
VA = 0
A
VB = 3
B
VA = 1
A
VB = 3
B
A
VA = 2
VB = 3
B
VA = 3
A
VB = 3
B
VA = 3
A
VB = 3
B
XM

Figure 13:

Both cars are initialized at their maximum speed of $V_{max} = 3$ and $D_{merge} = 3$.
In some cases, two cars arrive at the same time at the merging point, or at any other position after x_M. We decide that the car on lane 1 will have the priority, it will be the car on lane 0 who will slow down to create a gap between them the size of D_{merge}. At $t + 7$ they enter a stationary regime with the distance between them equal to D_{secu}.

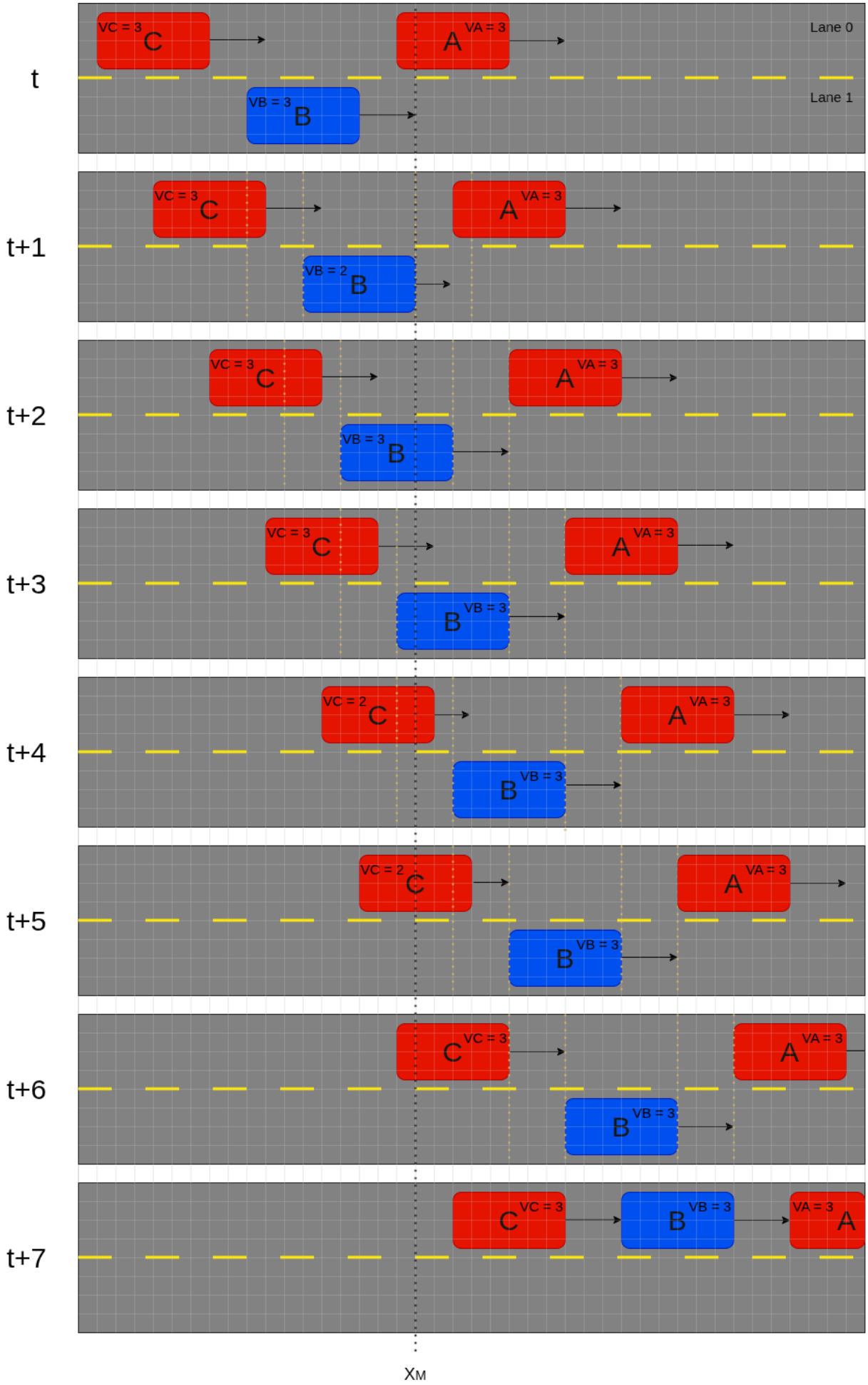


Figure 14:

All the cars are initialized at their maximum speed of $V_{max} = 3$ and $D_{merge} = 3$.
This configuration is a mix of figures 11 and 12. With the cars "A" and "C", too close to allow "B" to merge when it reaches the merging point. Like in figure 11, when the car in lane 1 reaches x_M it slows down to open the gap in front. But it makes the distance between "C" and "B" even shorter. Car "C" does not slow down to increase the gap between "C" and "B" until it passes the merging point. Because before x_M "C" is computing is speed based on car "A". Only once it reaches the merging point will it enter the virtual lane and base it's speed on "B".
This effect is reinforced when the lanes are filled with more cars. In the next figure, cars "A" and "B" are initialized exactly the same as figure 13, and will behave exactly the same. The only difference is that we added infinitely more cars in each lane to see the effects. At time t each lane is initialized in stationary regime with all the cars at $V_{max} = 3$ and the distance between them is equal to D_{secu}, meaning each lane is at it's maximum flow for V_{max}. The illustration is pushed until $t + 13$, so it is presented in two figures 15 and 16. Remember that cars "A, B" will behave exactly like in figure 13, the added cars are only here to illustrate at a larger scale how the merging algorithm works.

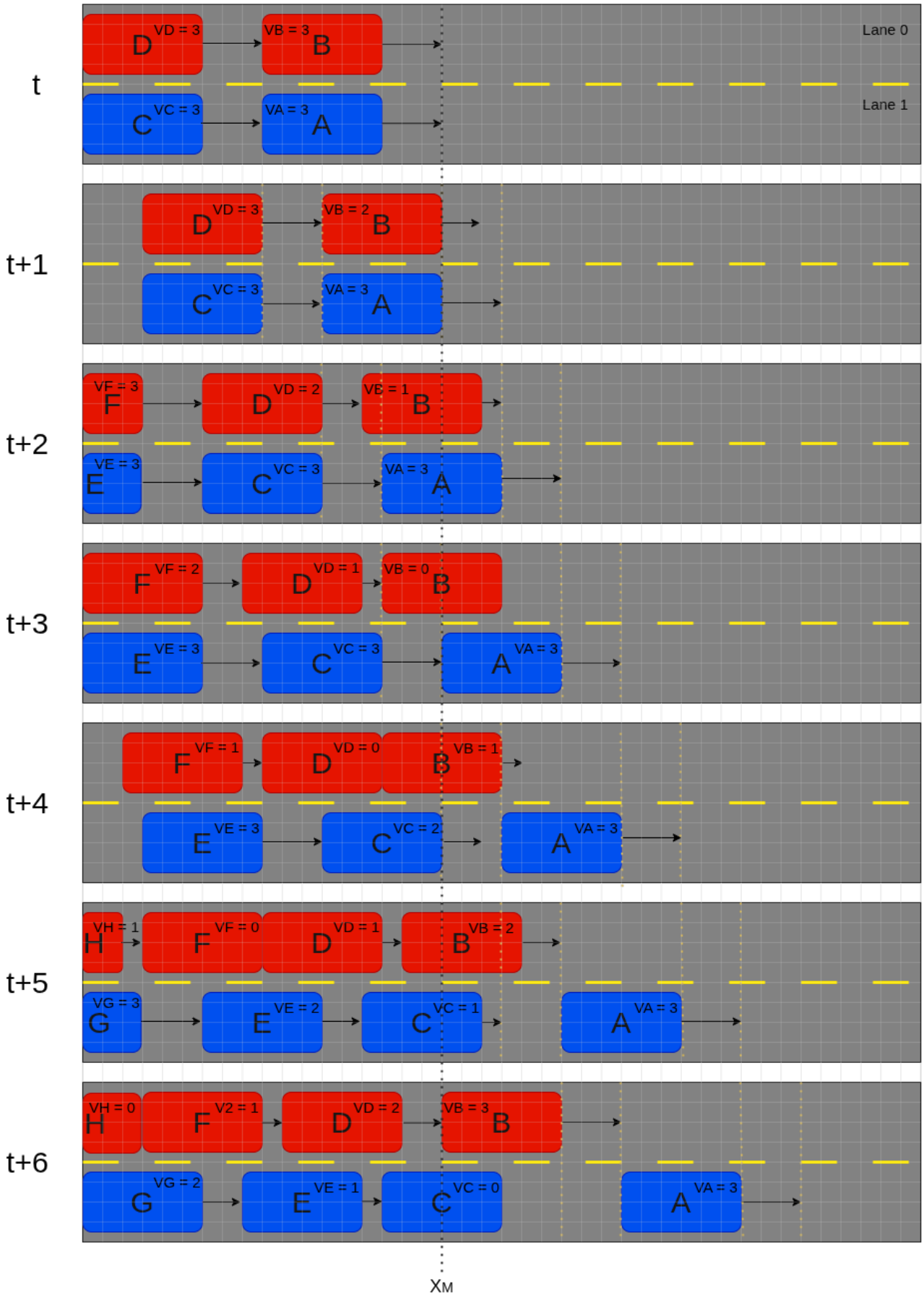

t
t+1
t+2
t+3
t+4
t+5
t+6
Lane 0
Lane 1
D VD = 3 B VB = 3
C VC = 3 A VA = 3
D VD = 3 B VB = 2
C VC = 3 A VA = 3
F VF = 3 D VD = 2 B VB = 1
E VE = 3 C VC = 3 A VA = 3
F VF = 2 D VD = 1 B VB = 0
E VE = 3 C VC = 3 A VA = 3
F VF = 1 D VD = 0 B VB = 1
E VE = 3 C VC = 2 A VA = 3
H VH = 1 F VF = 0 D VD = 1 B VB = 2
G VG = 3 E VE = 2 C VC = 1 A VA = 3
H VH = 0 F V2 = 1 D VD = 2 B VB = 3
G VG = 2 E VE = 1 C VC = 0 A VA = 3
XM

Figure 15:

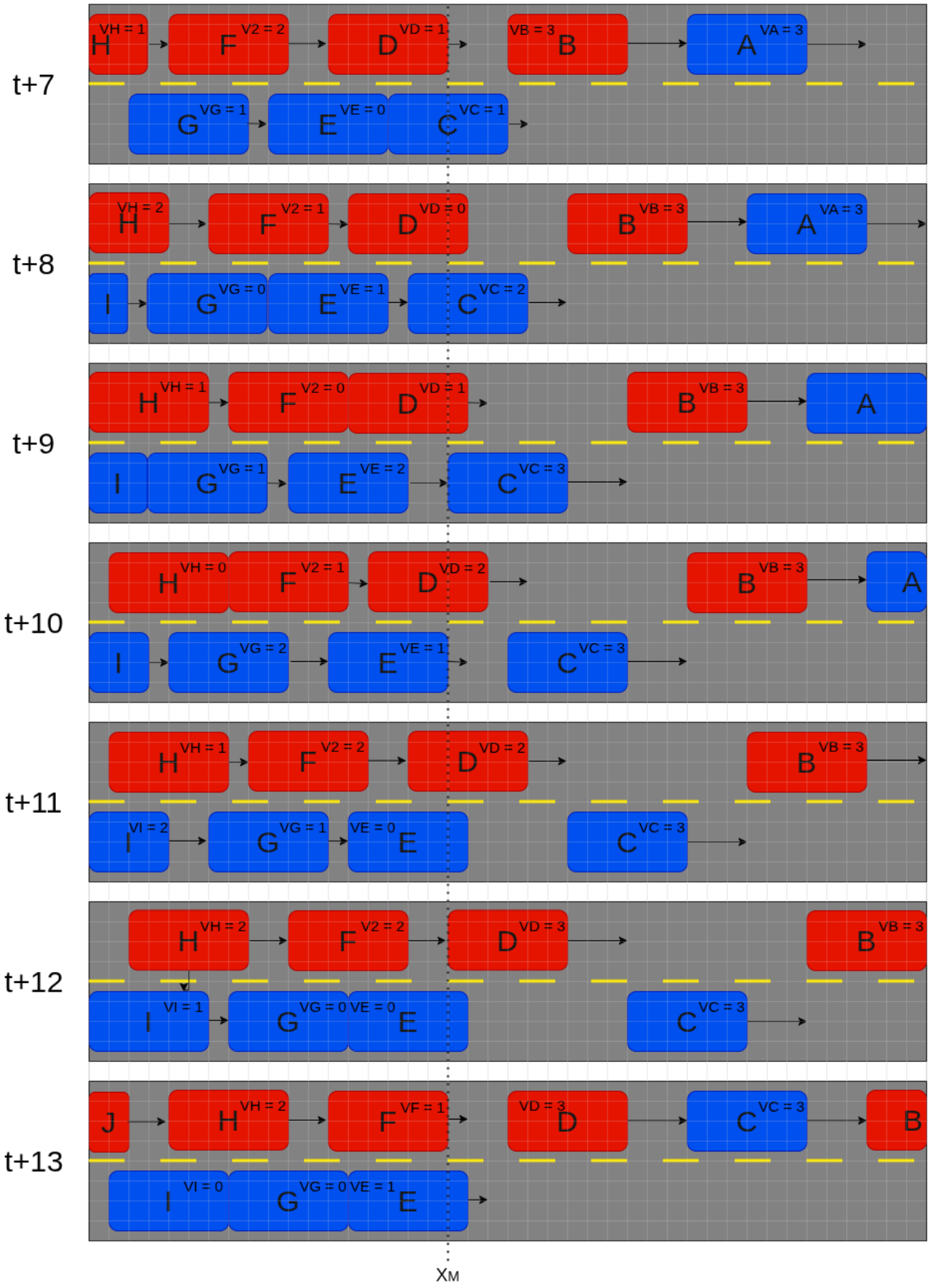


Figure 16:

We see that cars "A, B, C, D" have successfully merged. If we continued the simulation the merging will be in the alphabetic order. The merging algorithm, merges cars from alternating lanes, which impedes a lane to be completely immobile

and not allowing cars to pass x_M. We also see that before the merging point cars modify their speed based on the car in front of them in the same lane, after x_M it bases it's speed on the car directly in front independently of the lane. It can be observed at $t+4$ for car "C", up to that point it was computing it's speed based on car "A" and at $t+4$ if it continued to base it's speed on "A" it would have kept it's speed of $v_c = 3$. But it did not, "C" based it's speed on car "B", which is much closer and the speed regulator decreased the speed of "C" to allow the distance between "C" and "B" to increase. We could say that at $t+4$ car "C" entered the virtual lane. This moment can be seen at other moments for different cars. Here is the list of every occurrence; ($t+1$ for "B"), ($t+4$ for "C"), ($t+7$ for "D"), ($t+10$ for "E"), ($t+13$ for "F"). At each of these moments the car stops computing it's speed with the car in front in the same lane and starts computing it's speed with closest car instead, "entering the virtual lane".
The merging algorithm allows multiple cars to start the merging sequence even if the cars in front have not finished merging. It can be seen in figure 15 multiple times, an example is at $t+5$, car "A" is creating the gap between it's self and car "B", and at the same time "C" is creating the gap with "B".
In all of the examples from figures 9 to 16, $D_{merge} = D_{secu}$ we could reduce the length of D_{merge} and study the effects but D_{merge} can not be larger than D_{secu}. Because in every configuration except the one in figure 7, the cars stabilize with a gap between them equal to D_{secu}. We can not force cars in a stable configuration to increase the gap between them. In order to do so, it would require to modify their speeds and the speeds are defined by the functions "ComputeLane0(ListCars)" and "ComputeLane1(ListCars)" which imlements the speed regulator. Since the speed regulator almost always stabilizes cars at D_{secu}, using $D_{merge} > D_{secu}$ would create gaps the size of D_{secu} but in most scenarios cars would not merge. If $D_{merge} < D_{secu}$ cars would simply merge earlier and finish creating the gap D_{secu} while in lane 0.

To summarize, when a car passes the merging point, it will base it's speed on the closest car in front, independently of the lane. This acts as if after x_M, all cars are virtually in a single lane, and thanks to the characteristics of the speed regulator (2.1), the cars will adapt their speed until D_{secu} is obtained. The function "Incorporation(ListCars, D_{merge})" will monitor the distances (D_{begin}, D_{front}) of cars in lane 1 until they are both equal or larger than D_{merge} to allow the car to merge into lane 0.

3 Observations

We saw at the end of figure 16, that in both lanes a traffic jam began to form. We can measure how this traffic evolves by defining the start and the end of the traffic jam. The tail of the jam, (x_{tail}), is the position where cars arrive at the congestion, they slow down from their max speed V_{max} to a speed lower than V_{jam}. The head of the jam, (x_{head}), is the position where cars finally get a speed higher than V_{jam}, and accelerate back to their V_{max}. The value of V_{jam} is not arbitrary. The OFROU, (Office fédéral des routes [OFROU]) [3], defines a traffic jam on the high way when cars have a speed lower to 10km/h for more than a minute. Since we are doing relatively simple simulations compared to real life conditions simply looking for cars under $V_{jam} = 3m/s = 10.8km/h$ is enough to consider a car in a traffic jam. These measurements allows us to monitor the evolution of these two fronts and calculating the size of the jam, (D_{jam}), by simply subtracting the position of the head by the position of the tail.

$$D_{jam} = x_{head} - x_{tail} \tag{20}$$

But before analyzing any jam we need to understand what conditions create a congestion in the first place. In our case the jam is created because in lane 0 at point x_M lane 1 is adding cars to lane 0, and the sum of cars is too large for what the lane can handle. To measure a flow, we count the amount of cars that pass in a lane per second at fixed position, [cars/sec]. To calculate the maximum flow a lane can tolerate, cars need to be going at their top speed and have the minimum distance between them, $D_{secu} = V_{max}$ and account for the size of the car.

$$\begin{aligned} Q_{max} &= \frac{V_{max}}{D_{secu} + L} \\ Q_{max} &= \frac{m/s}{m+m} = \frac{1}{s} = s^{-1} \end{aligned} \tag{21}$$

This gives us the maximum flow allowed in a lane, in [cars/sec], and is the same Q_{max} for every lane. We call Q_0 the flow for lane 0 and Q_1 for lane 1. This two flows are called input flows, ($Q_{in} = Q_0 + Q_1$), meaning they are located before the merging point x_M. As a result of the merging, we will obtain an output flow called Q_{out}, which is measured at a position where all cars are in lane 0, so $Q_1 = 0$, which makes the output flow equal to the flow of lane 0, $Q_{out} = Q_0$.
So in order to create a traffic jam we need to have the input flow larger than the output flow $Q_{in} > Q_{out}$. If we assum the merging algorithm perfectly merges the lanes, loosing no time and space, when the input flow is higher or equal to input flow, $Q_{in} \geq Q_{out}$, the output should be equal to the maximum flow $Q_{out} = Q_{max}$. We will study what Q_{out} our algorithm can deliver compared to a perfect merging sequence.
The first scenario we will study is with $Q_{in} = \frac{Q_{max}}{2}$ which means that on lane 0 and 1 the flow is equal to $Q_0 = Q_1 = \frac{Q_{max}}{4}$.

Cars will be initialized with $V_{max} = 28m/s = 100.8km/h$, L = 5m, $x_M = 0m$ which gives us a initial flow of :

$$Q_0, Q_1 = \frac{Q_{max}}{4} = \frac{V_{max}}{4(D_{secu} + L)}$$

$$Q_0, Q_1 = \frac{28}{4(28+5)} = \frac{28}{132}$$

And to determine the distance, D, between cars to respect the flow of Q_0, Q_1.

$$\frac{28}{132} = \frac{28}{D+5}$$

$$7(D+5) = 924$$

$$D + 5 = 132$$

$$D = 127$$

With $D = 127$ the flow of Q_0 and Q_1 is respected when the cars will be initialized in a stationary regime, ($Q_0, Q_1 = \frac{28}{127+5} = \frac{28}{132}$). We have $Q_0 + Q_1 = Q_{in} = \frac{Q_{max}}{2}$, if we had a perfect algorithm the output should be the same as the input, $Q_{out} = Q_{in} = \frac{28}{132} cars/s$. We will initialize cars side by side like this :

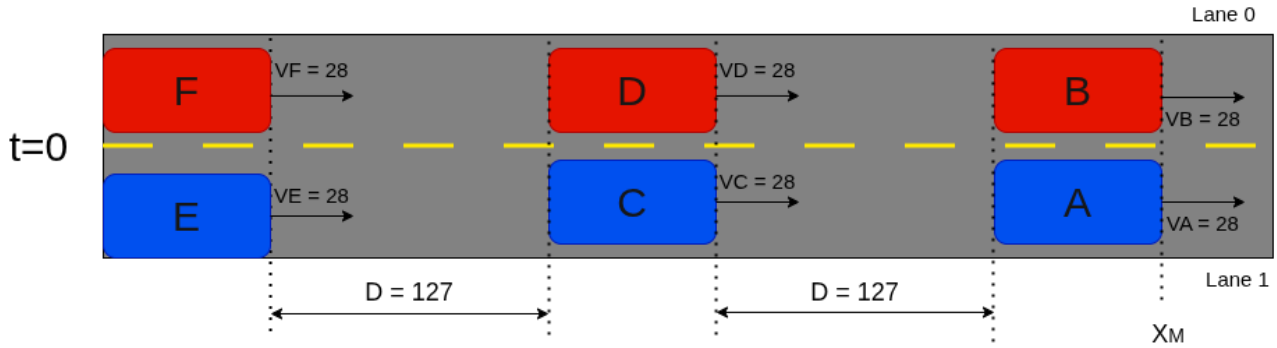


Figure 17: Cars are initialized with same configuration on both lines. $V_{max} = 28$, $D_{merge} = V_{max}$, $D = 127 > D_{secu} = 28$, $V_{min} = 0$, $x_M = 0$, $Q_0, Q_1 = \frac{28}{132}$, so the total input flow is $Q_{in} = Q_0 + Q_1 = 14/33 \approx 0.4242$cars/s.

After 1000 seconds of simulation we observe this flow :

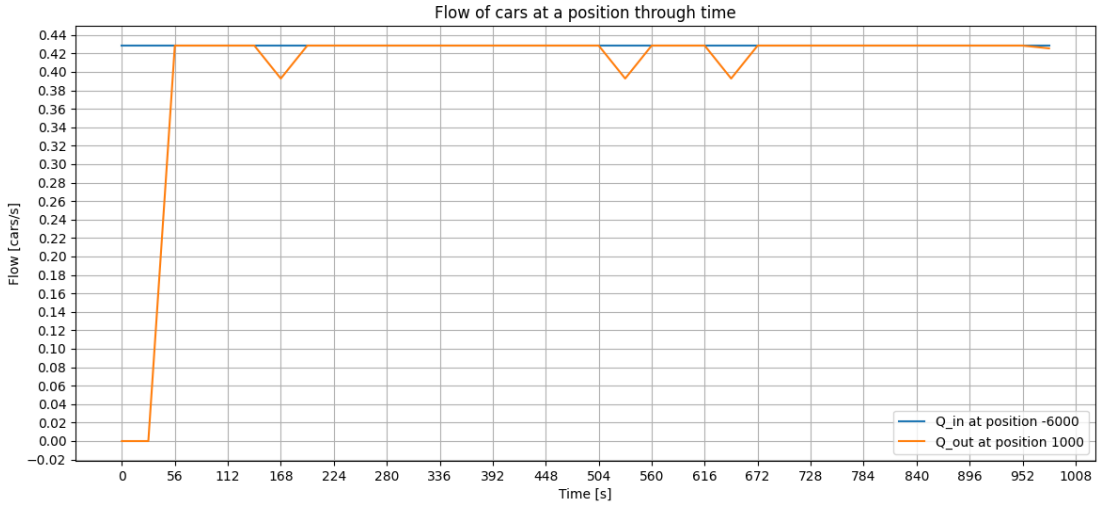


Figure 18: The flow is measured by counting the amount of cars that pass at position -6000 or 1000. The program counts the number, N, of cars that pass every 28s and divides it by 28, flow=$N/28$ [cars/s]. Which can explain little fluctuations.

The flow output flow is equal to the input flow, $Q_{in} = Q_{out} = \frac{14}{33} \approx 0.4242$. This means the flows are low enough to allow the merging algorithm to merge cars without impacting the flow. We will now double the input flow to obtain $Q_{in} = Q_{max}$. We keep all the other variables the same to see the sole impact of a larger input flow. So the flow of lane 0 and 1 stay equal to each other and the initial positions too like in figure 17, but with shorter distance between cars to respect $Q_{in} = Q_{max} = \frac{28}{33} \approx 0.8485$. The distance between cars will be $D = 61$.

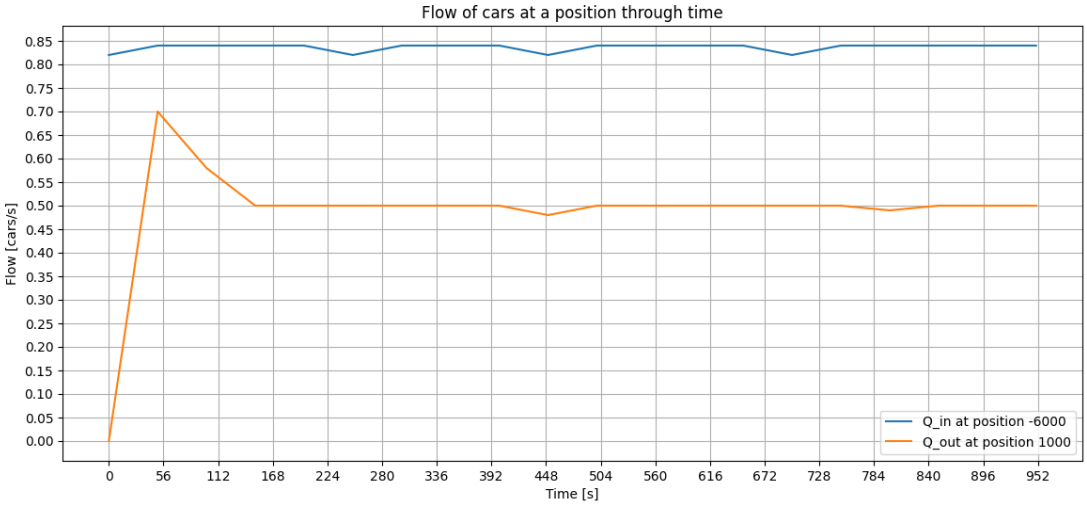


Figure 19: The input flow is $Q_{in} = Q_{max} = \frac{28}{33} \approx 0.8484$ cars/s.

We can see that the output flow is lower than the input flow. In perfect condition it would have stayed the same since $Q_{in} = Q_{max}$ but our algorithm demands to slow down the flow to allow cars to merge and the output flow stabilizes at $\approx$ 0.5 cars/s. We will now make double Q_{in} again and so now both lanes are at max flow, $Q_0, Q_1 = Q_{max}$. Which makes the input flow $Q_{in} = 2Q_{max} = \frac{56}{33} \approx 1.697$. The same simulation will be done again to see the performance of the merging algorithm.

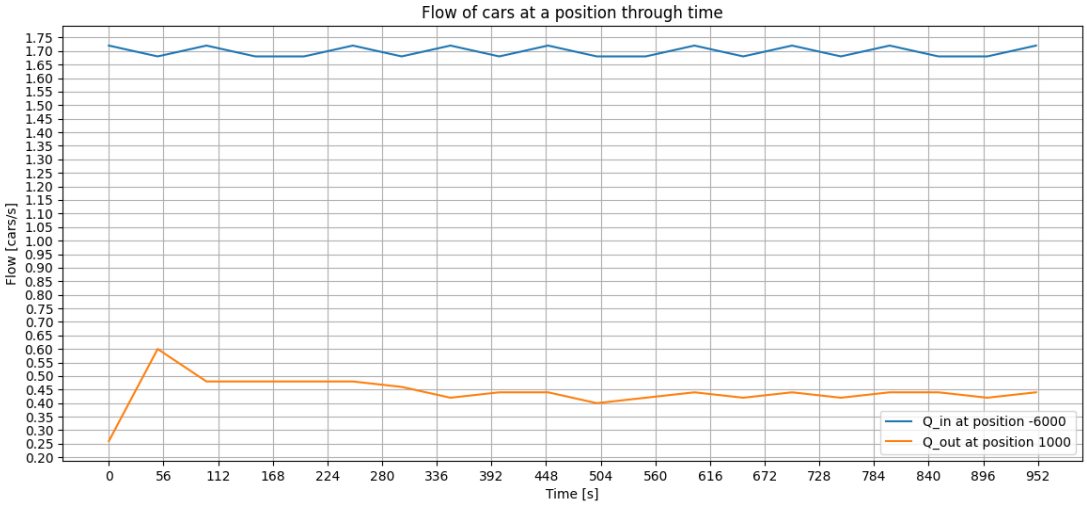


Figure 20: The input flow is $Q_{in} = 2Q_{max} = \frac{56}{33} \approx 1.697$ cars/s.

In the best case scenario Q_{out} would have been equal to Q_{max} but our algorithm has an output flow of that varies between 0.4 and 0.45, we approximate it to 0.425 [cars/s].

We will now make various simulation varying the input flow from $\frac{Q_{max}}{2}$ to $2Q_{max}$ and show the output flow the algorithm stabilizes at. We will use the same variables as the examples in figures 18,19,20 : $V_{max} = 28m/s$, $D_{merge} = V_{max}$,

$x_M = 0$, $V_{min} = 0$ and the initial configuration will be the same as figure 17, with the distance D adjusting to the input flow. So we have cars side by side.

Ratio to Q_in	Qmax/2	Qmax/1.75	Qmax/1.5	Qmax/1.25	Qmax/1.23	Qmax/1.20	Qmax/1.14	Qmax/1.10	Qmax/1.05	Qmax	1.38Qmax	1.5Qmax	1.75Qmax	1.83Qmax	2Qmax
Input flow	0.424	0.485	0.566	0.679	0.691	0.709	0.747	0.767	0.812	0.848	1.167	1.273	1.474	1.556	1.697
Output flow	0.424	0.485	0.566	0.67	0.67	0.67	0.67	0.67	0.5	0.5	0.5	0.5	0.5	0.43	0.42
Error (%)	0.0	0.0	0.0	1.33	3.04	5.5	10.31	12.65	38.42	41.04	57.16	60.72	66.08	72.37	75.25

Figure 21: The first row is the ratio of the input flow compared to the maximum flow, ($Q_{in} = X \cdot Q_{max}$). The second row is that same value but numeric. The third row is the output flow that the merging algorithm produced. The last row is the error between the input flow and the output flow.

We see that if the input flow is lower than $\frac{Q_{max}}{1.5}$, the merging has no impact on the resulting output flow. The more the flow increases the worst is the output further from the input. The output can not get any worse that 75.25%, because both lanes are at Q_{max}.
These simulations where all done with the flows of lane 0 and 1 equal, ($Q_0 = Q_1$). The initial positions of the cars where side by side like seen in figure 17. Now we will run the same simulations but with unequal flows between lanes. We will evaluate the same input flows as tested in figure 21, but this time $Q_0 = 2Q_1$. The sum of Q_0 and Q_1 will remain Q_{in} like in the precedent table. The first car of each lane will be initialized side by side but since the flows are not equal the following cars will not necessarily side by side. All other variables are have the same values as before, same V_{max}, V_{min}, D_{merge} etc. In this scenario with $Q_0 = 2Q_1$, lane 0 has more cars than lane 1. We will also have simulate with the flows between lanes inverse, $Q_1 = 2Q_0$. We want to see if merging algorithm is symmetric. Does merging $Q_0 = 2Q_1$ the same as merging $Q_1 = 2Q_0$?

Ratio to Q_in	Qmax/2	Qmax/1.75	Qmax/1.5	Qmax/1.25	Qmax/1.23	Qmax/1.20	Qmax/1.14	Qmax/1.10	Qmax/1.05	Qmax	1.38Qmax	1.5Qmax	1.75Qmax	1.83Qmax	2Qmax
Input flow	0.424	0.485	0.566	0.679	0.691	0.709	0.747	0.767	0.812	0.848	1.167	1.273	1.474	1.556	1.697
Output flow	0.424	0.485	0.566	0.46	0.48	0.67	0.66	0.5	0.5	0.5	0.5	0.43			
Output inverse flow	0.424	0.485	0.566	0.5	0.68	0.68	0.5	0.5	0.5	0.5	0.5	0.5			
Error OUT (%)	0.0	0.0	0.0	32.25	30.54	5.5	11.65	34.81	38.42	41.04	57.16	66.22			
Error OUT inverse (%)	0.0	0.0	0.0	26.36	1.59	4.09	33.07	34.81	38.42	41.04	57.16	60.72			

Figure 22: The first row is the ratio of the input flow compared to the maximum flow, ($Q_{in} = X \cdot Q_{max}$). The second row is that same value but numeric. The third row is the output flow that the merging algorithm produced, with $Q0 = 2Q_1$. The fourth row is the output flow that the merging algorithm produced, with the flows swapped in comparison with row three, $Q1 = 2Q_0$. The fifth and sixth row are the errors between the input flow and the output flow.

The last three columns are empty because of the condition that a lane must have double the flow. At $1.75Q_{max}$ and onward, the high flow lane would have to initialize cars that do not respect the minimum security distance, $D_{secu}0$. Analyzing such configuration do not make sense and as the characteristics of the speed regulator would force cars to increase the distance between them to D_{secu} and we would not be in a configuration with a lane that has double the flow of the other one. Looking at the results we see that the flows swapped between lanes do not significantly change the performance of the merging in relation to them. If we compare it to figure 21, equal flows between lanes has less loss in output flow than asymmetric flows. But if lane 0 has a larger flow or lane 1 has the larger flow the results do not vary much. It seems symmetric, except for the case where the input flow is $Q_{max}/1.23$. If we check the initializing position for $Q_0 = 2Q_1$ and $Q_1 = 2Q_0$.



Figure 23: $Q_1 = 2Q_0$, the input flow is $Q_{in} = Q_{max}/1.23$



Figure 24: $Q_0 = 2Q_1$, the input flow is $Q_{in} = Q_{max}/1.23$

Figure 23, is the scenario where the output flow is closest to the input flow. This happens because the distance between cars in lane 0 is equal to $2D_{secu} + L$ and on lane 1 is exactly positioned in the middle of that gap so when it passes the merging point the minimum security distance is verified right away and allows the car to switch lanes immediately without slowing down thus not slowing cars behind him neither on lane 0 or 1. The cars that are side by side do indeed have to slow down but since cars mentioned before do not impact the lane it is as if we had two lanes merging that had similar flows and the sum is lower than $Q_{max}/1.23$. This scenario can not happen in figure 24, because the cars are on lane 1 are never exactly in the middle of a gap the size of $2D_{secu}+L$, since the flow of lane 0 is double what it is in figure 23. This scenario is possible due to an extremely specific configuration and should happen often especially if we initialize cars with a nonhomogeneous density. Even if it would come to happen this particular configuration simply improves the merging sequence which is not a concern, since we want to optimize the output flow.

4 Conclusion

To summarize the observations made, this discrete simulation implements a solution to the problem of two lanes merging. It uses a merging sequence that alternates from which lane a car may incorporate the flow of cars on lane 0. We observed how the output flow looses it's performance while the input flow grows. But the merging never comes to halt even when lanes 0 and 1 are at full capacity. We also covered the cases where the the flow in lanes were not balanced. We saw that when we had a unbalance of flow between lanes. The performance was generally worse than with a balanced flow. But the main conclusion found, is that the performance of unbalanced flows, ($Q_0 = 2Q_1$ or $Q_1 = 2Q_0$), are the same regardless of which lane had the bigger flow. This is result comes from the fact that the merging algorithm by design, will alternate from which lane it pulls cars to make the merging. This result is based on observation with lanes having twice the flow of the other but it could be generalized to larger or smaller ratios.

This work grazes the complexity of the merging problem. We could continue the analysis of unbalanced flows to find when does the difference of flows begin to loose too much performance. We could have searched where the limit for different unbalanced ratios the performance begins to drop. We have not studied all the other variables, ($V_{max}, V_{min}, D_{merge}, V_{jam}$), and find if the configuration that optimizes the flow for balanced flow are the same for unbalanced flows.

References

[1] Bastien Chopard and Pierre Leone. "Global dynamics of lane merging algorithms for autonomous vehicles". In: *Natural Computing* (2026).

[2] Nathan Cohen, Bastien Chopard, and Pierre Leone. "Optimal safe driving dynamics for autonomous interacting vehicles". In: *Natural Computing* (2023).

[3] Office fédéral des routes (OFROU). *Définitions*. n.d. URL: `https://www.astra.admin.ch/astra/fr/home/themes/routes-nationales/fluidite-du-trafic-et-embouteillages/definitions.html` (visited on 06/29/2026).